THE LAWS OF SOFTWARE TEAMS

Zawinski's law

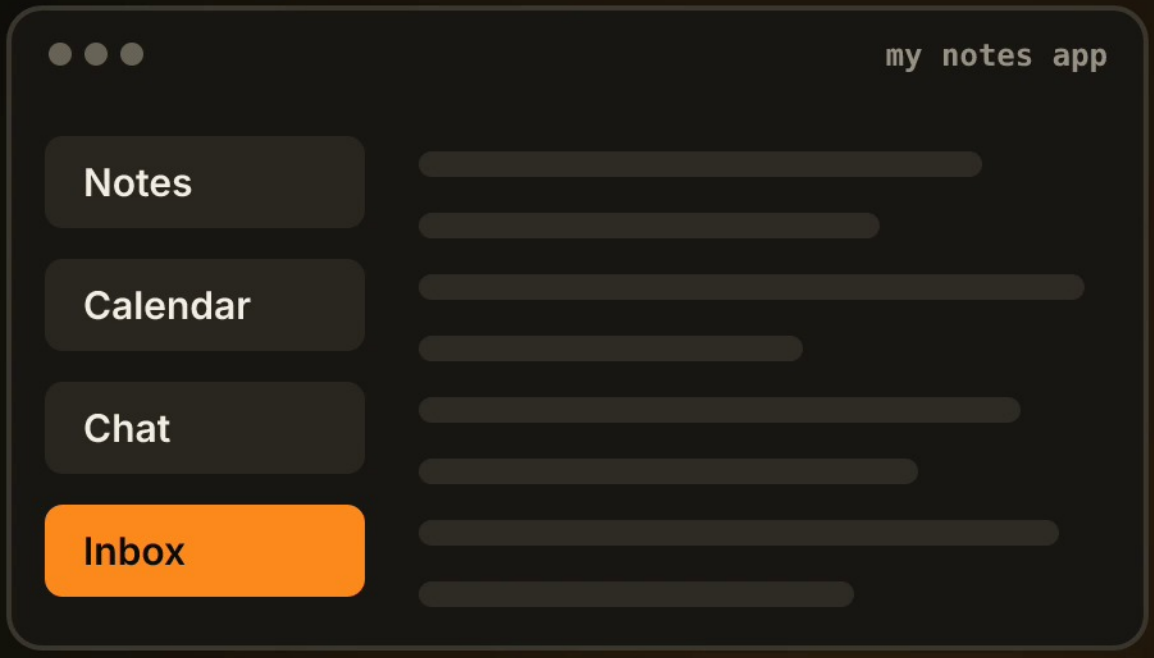
Every program attempts to expand until it can read mail

swipe →

Ivan Rykov

01 THE HOOK

Every app wants to read mail



This pattern has a name

illustration

02 THE LAW

Jamie Zawinski's law

"Every program attempts to expand until it can read mail"



THE SECOND HALF

"Those programs which cannot so expand are replaced by ones which can"

He called it the Law of Software Envelopment · jwz.org, the Jargon File

A browser grows a mailbox

Netscape · Zawinski worked on Netscape Mail

1994	Navigator 1.0
v2.0	Navigator 2.0 · + mail · + news
1997	Navigator · Messenger · Collabra · Composer · Address Book · Netcaster · Conference · Calendar

From a browser to an eight-app suite

Netscape Communicator, June 1997 · v2.x bundled mail and news (Wikipedia)

Not copycats
Platforms

JAMIE ZAWINSKI · 2020

"My point was not about copycats,
it was about platformization"

text editor → IDE → OS

Apps people live in all day
feel pressure to do everything

his 2020 clarification, as quoted on Wikipedia

Still true in 2026

Notion · the notes and docs app

APR 2025
Notion Mail launches



SEP 2026
The mail app closes



INSTEAD
Email moves into AI agents

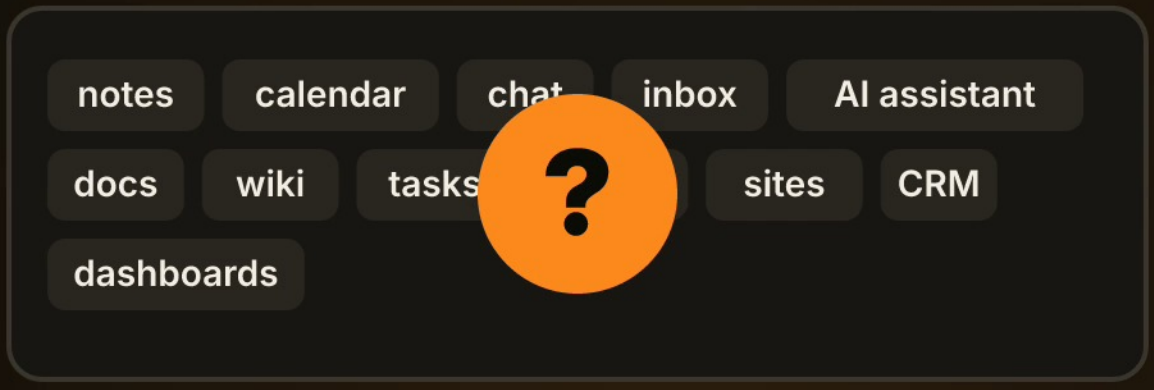


It still reads mail

TechCrunch, 15 Apr 2025 · Notion, closure announced 25 Jun 2026

Every yes adopts a child

"...you're adopting a child" · Getting Real, 37signals



New users get lost in a product that does everything

feature names illustrative

Do not start with yes

"Can we add chat?"

NOT NOW

WHAT WE WILL NOT BUILD

- × our own mail client
- × a finance module
- × a report builder

"Export to Excel?" ×3

Comes back again? Take a deeper look

based on Getting Real, 37signals · examples illustrative

Every product wants to read mail



Decide on purpose if yours should

WHAT I TAKE FROM THIS

When a new idea comes in, I do not start with yes or no. I think it through:

1. **The product stays focused on the main problem it was built to solve. We use MoSCoW top down, for the whole product and not only for features: what it must have, and what does not fit its purpose. "It would be nice to have" is not a reason to add something**
2. **What goes into the product is a dialogue and a compromise. The product team mainly owns what goes in, design how it feels, engineering how it is built and at what cost**
3. **Pushing back is part of the job. A no comes with context: which problem the request solves, what it costs, and an alternative. The goal is agreement, and a big disagreement goes one level up**

WHAT I TAKE FROM THIS

4. **Every request gets a priority, and incidents and critical bugs are fixed at once. "Not now" is not a hidden "no": what makes sense goes to the backlog, and we return to it after the current work. Not every idea goes to the backlog, so the backlog does not keep growing. A customer request that comes back again and again gets more weight**
5. **If testing a small hypothesis from the request is more important than the current work, we take it right after the current milestone. Nobody drops a task halfway for a new idea, because half-done tasks pile up and people burn out**

WHAT I TAKE FROM THIS

6. **A new feature joins the core flow or stays a separate part, depending on how closely they are linked and whether one can be used without the other**
7. **A feature that nobody uses and that has no future first goes behind a feature toggle, and later, ideally, out of the product and the code. Extra code takes developers' time. With AI agents, extra code costs less time, so we can keep it a little longer, but we still watch it**

Sources: jwz.org · The Jargon File · Wikipedia (Zawinski, 2020) · TechCrunch, 2025 · 9to5Mac, 2026 · Getting Real, 37signals

next: Brooks' law